

Day 5 Theory: Hosting, Render and What Comes Next

Workshop: Web Technology, Hands-on Full-Stack Workshop

Audience: BCA Second Year, Mandsaur University

Time for this day: 2 hours in class, plus about 20 minutes of reading before class

How to use this document

Read it once before class. Do not worry if some parts feel new. Write down your questions and bring them to class.

For four days we built the Notes Board. It runs on **your** computer. Today we move it to a computer on the internet, so that anyone with a link can open it. We also learn an honest lesson: the free hosting we use today has limits, and you must know them.

A note about facts. Websites like Render change their screens and rules from time to time. The facts about Render in this document come from Render's documentation, which was read on 6 October 2026. If your screen looks different from the lab sheet, trust your screen and ask your instructor.

1. Recap of Day 4

- Notes are saved in a **database** (SQLite). The database is one file, `notes.sqlite`, next to our server code.
 - The code is tracked by **Git** and stored on **GitHub**.
 - A password is needed to add or delete a note. It is kept in a `.env` file that is never uploaded to GitHub.
-

2. What is hosting?

A **server** is a computer that answers requests from browsers. Until now, your own computer was the server. When you opened `http://localhost:3000`, your browser talked to your own computer.

Hosting means renting a computer that is always connected to the internet, and running your app on it. The company that gives you this computer is called a **hosting provider**. Today we use a provider called **Render**.

Deploying means taking your code from your computer (or from GitHub) and starting it on the hosting provider's computer.

Why can't friends open my localhost link?

`localhost` means "this same computer". If you send `http://localhost:3000` to a friend, their browser looks at **their own** computer, not yours. Nothing is running there, so it fails.

	<code>localhost</code>	Public URL
Example	<code>http://localhost:3000</code>	<code>https://your-name.onrender.com</code>
Who can open it	Only you, on your computer	Anyone with the link
Where the app runs	Your computer	The hosting provider's computer
When it stops	When you close the terminal or switch off the computer	When the provider stops it, or you stop it

A **URL** is a web address. A **public URL** is an address that works from anywhere on the internet.

Could I host it on my own computer?

You could, but your computer would have to stay on all the time, with a fixed internet address and a safe setup. That is hard for a beginner. A hosting provider does this work for you.

3. HTTP and HTTPS

On Day 1 we saw that a browser sends a request and a server sends a response. That conversation uses a set of rules called **HTTP**.

HTTPS is HTTP with **encryption**. Encryption scrambles the data, so that other people on the network cannot read it while it travels. This matters a lot for our app, because you type a **password** into the page.

How to check:

- The address starts with `https://`.

- The browser shows a **padlock** icon near the address.

Our localhost link uses `http://` because the data never leaves your computer. A public link should use `https://`. Render's `onrender.com` links are expected to use `https://`. Always look at the address in the browser and check it for yourself.

4. What is Render?

Render is a hosting provider. We use its **Web Service** type, which runs a program that listens for requests, like our Express server.

How the deploy works

```
Your computer --git push--> GitHub --Render reads the code--> Render's computer
```

1. You **push** your code to GitHub (you learned this on Day 4).
2. You tell Render which GitHub repository to use.
3. Render copies the code onto its computer.
4. Render runs the **Build Command**. For us it is `npm install`, which downloads Express and the other packages.
5. Render runs the **Start Command**. For us it is `npm start`, which runs `node server.js` (because our `package.json` says so).
6. Render gives you a public URL that ends in `onrender.com`.

The messages from steps 3 to 5 are shown in a page called the **log**. When something goes wrong, the log tells you why. Reading the log is the most important skill of deployment.

The Render form fields we use

Field	What we write	What it means
Name	A name you choose, for example <code>yourname-notes-board</code>	Becomes part of your public URL
Region	A region near India if the list has one	The place where the computer is. Choose any available one if you do not see a near one
Branch	<code>main</code> (or the branch you pushed)	Which branch of your repository Render uses
Language	<code>Node</code>	The tool Render needs to run our code
Build Command	<code>npm install</code>	Runs once, before the app starts
Start Command	<code>npm start</code>	Starts the app
Instance type or plan	Free	No payment

The button labels and the order of the fields on the real screen are **not verified**. Use the lab sheet as a guide and read your screen.

Which Node.js version will Render use?

Render's documentation says the Node version can be set in four ways. If more than one is set, this order decides: the `NODE_VERSION` environment variable first, then a `.node-version` file, then a `.nvmrc` file, then the `engines` field in `package.json`. The documentation also advises giving an **upper limit** to the version range.

Our `package.json` has this:

```
"engines": {
  "node": ">=24 <25"
}
```

It means "Node 24 or newer, but lower than 25". It has an upper limit, as the documentation advises. The documentation also says that the default Node version for services created from 17 September 2026 is 24.21.0. Which exact version your service uses is something you can read in the deploy log. It is **not tested** by us in advance.

5. Small code changes for hosting

Our app needs three small changes before we deploy.

5.1 Listen on `0.0.0.0` and on the `PORT` number

Render's documentation says a web service must **bind** to the host `0.0.0.0` and should use the `PORT` environment variable. The default on Render is 10000.

- **Bind** means "start listening on an address".
- `0.0.0.0` means "accept visitors from any address", not only from this computer.
- A **port** is a numbered door on the computer (we met this on Day 2).
- The hosting provider **chooses** the port number and tells our app in the `PORT` setting. So we read it with `process.env.PORT`. On our own computer, the number is 3000.

5.2 A `/health` route

A `/health` route is a tiny page that answers `{"ok":true}`. It says "I am alive". It has no password and no data. We use it to test quickly that the server is up. Hosting services can also use a path like this to check an app, but we do not set that up today.

5.3 A `README.md` file

A **README** is the front page of a repository. GitHub shows it under the file list. It tells visitors what the project is and how to run it. A good README is part of professional work.

5.4 Check `package.json`

Two parts matter for hosting:

- `scripts.start` is `node server.js`. Render's Start Command `npm start` runs this.
- `engines.node` is `>=24 <25`. It tells Render which Node version we want.

6. Environment variables

An **environment variable** is a setting that lives **outside** your code. The program reads it when it starts. In Node.js we read it with `process.env.NAME`.

Our app needs `NOTES_PASSWORD`. On your computer it comes from your `.env` file. On Render there is no `.env` file, because we never upload it. So we type the password into the Render dashboard, in the **Environment** settings of the service (you can also find it under **Advanced** when you create the service).

Why this is the right way:

- The password is not in your code, so it is not in GitHub.
- You can change the password later without changing code.
- Render keeps it as a setting of your service.

If you forget this step, our server stops at start and writes `NOTES_PASSWORD is not set.` in the log. This is on purpose. A server that starts without a password would be unsafe.

7. The honest lesson: what the Free plan can and cannot do

Render's Free web service is good for learning. It is **not** good for a real product. Render's documentation says:

Fact	What it means for our Notes Board
Ephemeral filesystem. Files written on a Free web service are lost when the service restarts, redeploys or spins down	Our <code>notes.sqlite</code> file is deleted . The notes disappear
Spin down. A Free web service spins down after 15 minutes without incoming requests, and takes about one minute to spin up again	The first visit after a quiet time is slow
750 Free instance hours per workspace per month	A limited amount of running time every month
Render might restart a Free service at any time	Data can be lost at any time, even while people use the app
No persistent disks on the Free plan	We cannot fix the problem by adding a disk on Free
No SSH or shell access on the Free plan	We cannot log in to the computer and look around. We only have the log

Ephemeral means "not lasting". The word describes a filesystem whose files do not last.

The "data reset" you will see in the lab

When the service starts, `data.js` creates the table, and if the table is empty it adds **3 starter notes**. On a Free web service the file is gone after a restart, so the table is empty again and the **same 3 starter notes appear**. Your own notes are gone. This is not a bug in your code. It is how the hosting works. We will do this on purpose in Part 8 of the lab, so that you see it with your own eyes.

What the real fixes are

1. **A hosted database.** Keep the data in a database service that is separate from the app, for example PostgreSQL. Render's documentation says a **free** Render PostgreSQL database **expires 30 days after it is created** (with a short grace period) and holds 1 GB. So it is fine for a trial, but **not for real use**. Paid database plans exist. We do not talk about prices here. Look at the provider's pricing page when you need it.
2. **A persistent disk.** This is a disk that keeps its files across restarts. Render's documentation says it is **not available on the Free plan**. It needs a paid plan.

We do not do either fix today. Our optional PostgreSQL track from Day 4 is the first step toward fix number 1. The rest is for you to explore after the workshop.

The lesson in one line: free hosting is for learning and demos. When people depend on your app, you need a database that is kept safe, and that usually costs money.

8. Security recap

1. **Password in an environment variable.** Never write a password inside your code. Anything in your code can end up in GitHub, and a public GitHub repository can be read by everyone.
2. **Never commit `.env`.** Our `.gitignore` file has a line `.env` so Git skips it. Check this before every push. If `.env` shows up in `git status` as a file to add, stop and ask.
3. **Check for `https://`.** Before typing a password into a page, look for `https://` and the padlock.
4. **`textContent`, not `innerHTML`.** We use `textContent` so that text typed by users is shown as plain letters and never runs as code. This stops the attack called XSS, which we met on Day 2. Today our app is public, so this protection matters more than before.
5. **If a password leaks, change it.** Suppose you pushed a password to GitHub, or you told it to a stranger. Do this:
 1. Change the password at once, in every place where it is used. For us, that is the `NOTES_PASSWORD` setting on Render and your `.env` file.
 2. Know that deleting the file or the line later does **not** make the old password safe. Git keeps history, and other people may already have copied it.
 3. Use a **new** password that you have never used anywhere else.
6. **Use a password that is only for this app.** Do not use the password of your email, your bank or your GitHub account.

9. What to learn next

This is a list of topics, in a sensible order. It is not a promise about any job or any salary. It is a map of what exists.

Topic	Why
More JavaScript	Everything in web development rests on it. Practise a little every day
A frontend framework (for example React)	Helps to build bigger pages with less repeated code
SQL	The language of databases. We used a little of it in <code>data.js</code>
Authentication with real accounts	Our app has one shared password. Real apps have users, with sign up, login and logout
Testing	Code that checks your code, so that changes do not break old features
Deploying and hosting	Databases that last, custom domain names, logs, and safe settings
Git and GitHub habits	Branches, pull requests and writing clear commit messages

Keep your repository. Add small features. A project that you built, understand and can explain is a good way to learn.

10. The showcase and the success tiers

At the end of the class, each student shows the work in a small group (a **pod**). You show either your live link or your localhost screen.

Tier	What you reached	Meaning
Tier 1	The Notes Board works on localhost	This is a pass. Everyone should reach this
Tier 2	The code is pushed to GitHub	You can show your repository
Tier 3	The app is live on Render	You have a public link

If Render sign-up does not work for your account, you stay at Tier 2. That is a good result. Your instructor will show one live deploy on the projector.

11. Glossary

Word	Meaning
Hosting	Renting a computer on the internet to run your app
Deploy	To put your app on a hosting provider and start it there
Render	The hosting provider we use today
Web Service	The Render type that runs a server like ours
Public URL	A web address that anyone on the internet can open
localhost	The name for "this same computer"
HTTPS	HTTP with encryption. The address starts with <code>https://</code> and the browser shows a padlock
Bind	To start listening on an address and a port
0.0.0.0	An address that means "accept visitors from anywhere"
Environment variable	A setting kept outside the code and read by the program when it starts
Build Command	The command that Render runs once to prepare the app (<code>npm install</code>)
Start Command	The command that Render runs to start the app (<code>npm start</code>)
Log	A page of messages from the build and from the running app
Ephemeral	Not lasting. Files on a Free web service do not last
Spin down	The service stops after a quiet time. The next visit starts it again
Persistent disk	A disk that keeps files after a restart. Not available on the Free plan
Instance hours	The hours your service is running. The Free plan has a monthly limit
Auto-deploy	Render deploys again by itself when you push new code
README	The front page of a repository
Pod	A small group of students in the showcase

12. Before class

1. Read this document once.

2. Make sure your Notes Board runs on your computer (`npm start` in the terminal, then open `http://localhost:3000`).
3. Make sure you can sign in to GitHub on the lab computer. You will use your GitHub account to sign in to Render. If you forgot your GitHub password, reset it before class.
4. Write down the questions you have.

Questions to think about

1. Why can your friend not open `http://localhost:3000` on your computer?
2. Why do we put `NOTES_PASSWORD` in the Render dashboard and not in `server.js` ?
3. Why did the 3 starter notes come back after a restart, and what would you change to stop your notes from disappearing?